PROJETO DE EXTENSÃO · FUNDAMENTOS DE PROGRAMAÇÃO

Try, Catch & Learn

< / >

# Caderno de Missões III

## Aprendendo a programar em linguagem C

- Fase 5 — Muitos Dados, Uma Só Caixa: Vetores e Matrizes
- Fase 6 — A Ficha Completa: Struct

**13 missões · 10 questões + 3 desafios**

De uma caravana de mercadores até um duelo final com fichas de jogador.

Nome do(a) programador(a): _____

Turma / Data: _____

# Bem-vindo(a) de volta, programador(a)!

O próximo grande salto na sua jornada de programação em C

Este é o terceiro caderno da nossa jornada em **C**. Até aqui você já sabe guardar valores, repetir tarefas e organizar o código em funções. Agora vamos resolver um problema novo: **o que fazer quando precisamos guardar muitos valores do mesmo tipo** (as notas de uma turma inteira) ou **várias informações diferentes sobre uma mesma coisa** (nome, idade e altura de uma pessoa)? É para isso que servem os vetores, as matrizes e as struct.

| Parte | O que você vai treinar | Missões |
|---|---|---|
| **Fase 5** | Vetores (1D), matrizes (2D), índices, percorrer com `for` | Questões 1 a 10 |
| **Fase 6** | `struct`, campos, operador ponto (`.`), structs com funções | Questões 1 a 10 |
| **Desafios** | Tudo junto: vetores, matrizes, structs, laços e funções | Desafios 1 a 3 |

⚡ **Regras de ouro (lembrete rápido)**

- Todo comando termina com `;`; nomes de variáveis diferenciam maiúsculas de minúsculas.
- Sem acentos dentro do `printf` — escreva `"Voce"` em vez de `"Você"` para evitar falhas de codificação.
- Compile com `gcc arquivo.c -o programa` e rode com `./programa` (Windows: `programa.exe`).
- O primeiro índice de um vetor ou matriz é **sempre 0**, nunca 1.

# Muitos Dados, Uma Só Caixa: Vetores e Matrizes

Vetores e matrizes, de forma organizada e eficiente

## 1. Por que precisamos de vetores?

Imagine guardar as notas de 30 alunos de uma turma. Sem vetores, seriam 30 variáveis separadas (`nota1, nota2... nota30`) e 30 parâmetros em cada função. Um vetor guarda vários valores do mesmo tipo sob um único nome, cada um numa posição numerada.

## 2. Declarando e entendendo o índice

SINTAXE BÁSICA — C

```c
tipo nome_do_vetor[tamanho];
float notas[5]; // reserva espaco para 5 numeros reais (uma "estante" com 5 gavetas)
```

Estrutura na Memória (Estante de Gavetas):

| [0] | [1] | [2] | [3] | [4] |
|---|---|---|---|---|
| 7.5 | 8.0 | 6.5 | 9.0 | 10.0 |

⚠️ **Atenção ao índice!**
A contagem sempre começa em **0**. Um vetor de tamanho 5 tem posições válidas de `[0]` até `[4]` — tentar acessar `[5]` é um erro clássico (acessa memória fora do vetor, o que pode travar o programa ou dar um resultado imprevisível).

## 3. Guardando e lendo valores

O índice entre colchetes acessa cada gaveta individualmente:

```c
idades[0] = 18;
idades[1] = 25;
printf("A primeira idade e: %d\n", idades[0]); // Exibe: 18
```

## 4. Vetores + laços + funções: a combinação perfeita

O `for` é ideal para percorrer todas as posições de um vetor de uma vez só, e podemos enviar o vetor inteiro para uma função processar:

```c
void exibirVetor(int vet[], int tamanho) {
    for (int i = 0; i < tamanho; i++) {
        printf("Posicao %d: %d\n", i, vet[i]);
    }
}
```

## 5. Matrizes: uma tabela de linhas e colunas

Try, Catch & Learn

Se o vetor é uma fita linear de valores, a matriz é uma tabela com linhas e colunas bidimensionais:

```
tipo nome_da_matriz[linhas][colunas];
int tabela[3][3]; // 3 linhas, 3 colunas (9 posicoes no total)
```

| Coordenada | Coluna 0 | Coluna 1 | Coluna 2 |
|---|---|---|---|
| **Linha 0** | `tabela[0][0]` | `tabela[0][1]` | `tabela[0][2]` |
| **Linha 1** | `tabela[1][0]` | `tabela[1][1]` | `tabela[1][2]` |
| **Linha 2** | `tabela[2][0]` | `tabela[2][1]` | `tabela[2][2]` |

## 6. Percorrendo uma matriz: dois for encadeados

O laço de fora escolhe a linha; o de dentro percorre as colunas daquela linha:

```
for (int i = 0; i < 2; i++) {       // i controla as LINHAS
    for (int j = 0; j < 3; j++) {   // j controla as COLUNAS
        printf("%d ", matriz[i][j]);
    }
    printf("\n");                   // pula linha ao terminar cada linha da matriz
}
```

### 7. Pensando como um programador

Antes de criar um vetor ou matriz, responda sempre a estas 3 perguntas fundamentais:

1. **Quantos dados e de qual tipo?** (ex.: 10 idades inteiras → `int idades[10];`)
2. **Quais são os índices válidos?** (lembre-se: de 0 até tamanho - 1)
3. **Os dados formam uma lista linear (vetor) ou uma tabela bidimensional (matriz)?**

## F5.1 As Caixas Numeradas

Preenchendo e exibindo um vetor

Um armazém de logística recebeu 5 caixas e precisa anotar o peso de cada uma numa única lista, para depois conferir tudo de uma vez.

> 💡 **Lembrete teórico:** Declare o vetor com `int numeros[5];`, preencha cada posição com um `for` usando `scanf("%d", &numeros[i]);` e exiba todas com outro `for`.

**Sua missão:**

1. Crie um vetor `numeros[5]`.
2. Use um `for` para ler os 5 números digitados pelo usuário.
3. Use outro `for` para exibir todos os números, um ao lado do outro.

EXEMPLO DE EXECUÇÃO

```
Digite o numero 1: 10
Digite o numero 2: 25
Digite o numero 3: 3
Digite o numero 4: 47
Digite o numero 5: 8
Voce digitou: 10 25 3 47 8
```

## F5.2 A Mão de Cartas

Vetor pré-preenchido e acesso por índice

Num jogo de cartas, a mão do jogador já vem com 6 cartas sorteadas. O programa precisa mostrar a mão inteira e também destacar o valor de cartas específicas, em posições escolhidas.

> 💡 **Lembrete teórico:** Um vetor pode já nascer preenchido na declaração: `int cartas[6] = {12, 45, 7, 90, 33, 61};`. Cada posição é acessada diretamente pelo índice, sem precisar de `scanf`.

**Sua missão:**

1. Declare o vetor `cartas[6]` já com os valores `{12, 45, 7, 90, 33, 61}`.
2. Mostre a mão inteira com um `for`.
3. Depois, mostre separadamente o valor das cartas nas posições 0, 3 e 5 (sem usar laço para essa parte).

EXEMPLO DE EXECUÇÃO

```
As cartas da mao sao: 12 45 7 90 33 61
Carta na posicao 0: 12
Carta na posicao 3: 90
Carta na posicao 5: 61
```

## F5.3 O Tesouro dos Oito Baús

Vetor + acumulador

Um grupo de aventureiros encontrou uma sala com 8 baús. Cada baú tem uma quantidade diferente de moedas, e o grupo quer saber o total do tesouro.

> 💡 **Lembrete teórico:** A combinação de vetor com acumulador é igual à da Fase 3, só que agora cada valor lido vai para uma posição do vetor em vez de ser descartado: `moedas[i] = valor; total = total + moedas[i];` (ou somar direto durante a leitura, como no gabarito).

**Sua missão:**

1. Declare o vetor `moedas[8]`.
2. Use um `for` para ler a quantidade de moedas de cada baú e já somar ao total.
3. Mostre o total de moedas do tesouro.

EXEMPLO DE EXECUÇÃO

```
Moedas no bau 0: 10
Moedas no bau 1: 5
Moedas no bau 2: 20
Moedas no bau 3: 8
Moedas no bau 4: 0
Moedas no bau 5: 15
Moedas no bau 6: 3
Moedas no bau 7: 9
Total de moedas do tesouro: 70
```

## F5.4 O Recorde do Campeonato

Busca do maior valor

Seis jogadores disputaram uma partida e o sistema precisa descobrir, entre as pontuações digitadas, qual foi a maior — o recorde da rodada.

> 💡 **Lembrete teórico:** Um padrão clássico: guarde o primeiro valor do vetor numa variável `maior` e, percorrendo o resto do vetor, atualize `maior` sempre que encontrar um valor ainda maior.

**Sua missão:**

1. Declare o vetor `pontuacoes[6]` e leia os 6 valores.
2. Inicie `maior` com `pontuacoes[0]`.
3. Percorra o vetor a partir da posição 1, comparando cada valor com `maior` e atualizando quando necessário.
4. Mostre a maior pontuação encontrada.

EXEMPLO DE EXECUÇÃO

```
Pontuacao do jogador 1: 340
Pontuacao do jogador 2: 580
Pontuacao do jogador 3: 210
Pontuacao do jogador 4: 999
Pontuacao do jogador 5: 450
Pontuacao do jogador 6: 120
A maior pontuacao foi: 999
```

## F5.5 A Semana Mais Fria

Busca do menor valor

Uma estação meteorológica registrou a temperatura de cada um dos 7 dias da semana e quer saber qual foi o dia mais frio.

> 💡 **Lembrete teórico:** O mesmo padrão da questão anterior, mas invertido: em vez de `if (valor > maior)`, usamos `if (valor < menor)`.

**Sua missão:**

1. Declare o vetor `termometros[7]` e leia as 7 temperaturas.
2. Encontre a menor temperatura da semana, seguindo a mesma lógica da Questão 4.
3. Mostre o resultado.

EXEMPLO DE EXECUÇÃO

```
Temperatura do dia 1: 18
Temperatura do dia 2: 15
Temperatura do dia 3: 9
Temperatura do dia 4: 22
Temperatura do dia 5: 11
Temperatura do dia 6: 14
Temperatura do dia 7: 17
A temperatura mais fria da semana foi: 9
```

## F5.6 A Despensa Mágica

Busca de um valor específico (com break)

Uma cozinha encantada guarda 5 ingredientes especiais, cada um identificado por um código. O cozinheiro quer saber se um ingrediente específico está disponível na despensa, sem precisar abrir todas as prateleiras manualmente.

> 💡 **Lembrete teórico:** Para procurar um valor dentro de um vetor, percorremos as posições comparando cada uma com o valor procurado. Ao encontrar, podemos usar uma "bandeirinha" (`encontrado = 1;`) e `break` para parar a busca imediatamente, sem continuar olhando o resto do vetor.

**Sua missão:**

1. Declare o vetor `ingredientes[5] = {7, 14, 2, 30, 9};`.
2. Leia o código do ingrediente procurado.
3. Percorra o vetor comparando cada posição com o valor procurado; se encontrar, marque `encontrado = 1` e pare a busca com `break`.
4. Mostre se o ingrediente está ou não na despensa.

EXEMPLO DE EXECUÇÃO

```
=== Execução: encontrado ===
Digite o codigo do ingrediente: 30
Ingrediente 30 esta na despensa!

=== Execução: não encontrado ===
Digite o codigo do ingrediente: 99
Ingrediente 99 NAO esta na despensa.
```

## F5.7 A Calculadora de Médias (Revisitada)

Vetor passado para uma função

A escola quer melhorar o sistema de médias da Fase 4: em vez de uma função que recebe notas separadas uma a uma, ela deve receber o vetor inteiro de notas de uma só vez.

> 💡 **Lembrete teórico:** Uma função pode receber um vetor como parâmetro: `float calcularMedia(float notas[], int tamanho)`. Dentro da função, um `for` percorre `notas[i]` normalmente, como se o vetor fosse local.

**Sua missão:**

1. Crie a função `float calcularMedia(float notas[], int tamanho)`, que soma todas as notas num `for` e devolve a média.
2. No `main`, declare `notas[4]`, leia os 4 valores e chame a função, mostrando o resultado.

EXEMPLO DE EXECUÇÃO

```
Nota 1: 8
Nota 2: 7.5
Nota 3: 9
Nota 4: 6.5
Media da turma: 7.75
```

## F5.8 O Relatório do Depósito

Vetor + procedimento

O gerente de um depósito quer um relatório organizado mostrando quantas caixas há em cada uma das 6 prateleiras — e a exibição deve ficar separada da leitura, num procedimento próprio (como na Fase 4).

> 💡 **Lembrete teórico:** Assim como funções podem receber vetores, procedimentos (`void`) também podem: `void exibirEstoque(int estoque[], int tamanho)`.

**Sua missão:**

1. Crie o procedimento `void exibirEstoque(int estoque[], int tamanho)`, que mostra `Prateleira <i>: <valor> caixas` para cada posição.
2. No `main`, declare `estoque[6]`, leia os valores e chame o procedimento para mostrar o relatório.

EXEMPLO DE EXECUÇÃO

```
Caixas na prateleira 0: 12
Caixas na prateleira 1: 5
Caixas na prateleira 2: 0
Caixas na prateleira 3: 8
Caixas na prateleira 4: 20
Caixas na prateleira 5: 3
--- Relatorio do deposito ---
Prateleira 0: 12 caixas
Prateleira 1: 5 caixas
Prateleira 2: 0 caixas
Prateleira 3: 8 caixas
Prateleira 4: 20 caixas
Prateleira 5: 3 caixas
```

## F5.9 O Tabuleiro Misterioso

Matriz 2×2 + soma total

Um jogo de tabuleiro simples tem uma grade de 2 linhas por 2 colunas, e cada casa guarda um valor digitado pelo jogador. O sistema precisa somar todos os valores do tabuleiro.

> 💡 **Lembrete teórico:** Para preencher uma matriz, usamos dois `for` encadeados (linha e coluna) com `scanf("%d", &matriz[i][j]);`, somando a cada leitura.

**Sua missão:**

1. Declare a matriz `tabuleiro[2][2]`.
2. Com dois `for` aninhados, leia o valor de cada casa e some num acumulador.
3. Mostre a soma de todas as casas do tabuleiro.

EXEMPLO DE EXECUÇÃO

```
=== TABULEIRO 2x2 ===
Valor em [0][0]: 4
Valor em [0][1]: 7
Valor em [1][0]: 2
Valor em [1][1]: 9
Soma de todas as casas: 22
```

## F5.10 O Mapa do Cinema

Matriz pré-preenchida + contagem condicional

Uma sala de cinema pequena tem 2 fileiras com 3 assentos cada. O valor 1 significa "ocupado" e 0 significa "livre". O sistema precisa desenhar o mapa da sala e contar quantos assentos estão ocupados e quantos estão livres.

> 💡 **Lembrete teórico:** Dentro do laço duplo que percorre a matriz, um `if` decide o que fazer com cada valor — exatamente como combinar decisão (Fase 2) com repetição (Fase 3), agora dentro de uma matriz.

**Sua missão:**

1. Declare a matriz `assentos[2][3]` já preenchida com `{{1, 0, 1}, {0, 0, 1}};`.
2. Com dois `for` aninhados, mostre `[X]` para assento ocupado e `[ ]` para livre, pulando linha ao final de cada fileira.
3. Conte e mostre o total de assentos ocupados e livres.

EXEMPLO DE EXECUÇÃO

```
=== MAPA DO CINEMA ===
[X] [ ] [X]
[ ] [ ] [X]
Assentos ocupados: 3
Assentos livres: 3
```

# A Ficha Completa: Struct

Agrupando informações relacionadas numa única estrutura

## 1. O que são variáveis compostas?

Até agora, cada variável guardava um único valor isolado. Mas uma ficha de pessoa tem nome, idade e altura — informações diferentes, porém relacionadas entre si, que fazem muito mais sentido juntas do que espalhadas em variáveis soltas.

## 2. Criando uma struct

A struct define o **formato do molde** da ficha:

```c
struct Pessoa {
    char nome[50];
    int idade;
    float altura;
};
```

`struct Pessoa` cria um novo tipo de dado com três campos: `nome`, `idade` e `altura`. O `;` depois da chave final é **obrigatório**. Definir a struct não guarda dados ainda — é apenas a planta da ficha.

## 3. Usando uma struct: o operador ponto

O operador ponto (`.`) acessa cada campo interno da variável struct:

```c
struct Pessoa pessoa1;
strcpy(pessoa1.nome, "Ana");
pessoa1.idade = 20;
pessoa1.altura = 1.65;
printf("Nome: %s\n", pessoa1.nome);
printf("Idade: %d\n", pessoa1.idade);
```

> ⚠️ **Atenção com texto em struct!**
> Um campo de texto, como `char nome[50];`, é um vetor de caracteres — ele não aceita atribuição com `=` depois da declaração. Use `strcpy(pessoa1.nome, "Ana");` (que exige `#include <string.h>`) para atribuir um texto fixo, e `scanf(" %[^\n]", pessoa1.nome);` para ler uma linha inteira digitada pelo usuário.

## 4. Por que usar struct?

- Agrupa dados que pertencem logicamente ao mesmo elemento.
- Deixa o código legível, modular e fácil de manter.
- Facilita representar entidades do mundo real: pessoas, produtos, alunos, personagens de jogos.

## 5. Vetores de struct e structs em funções

Uma struct pode ser colocada dentro de um vetor (`struct Pessoa turma[30];` para 30 fichas) e pode ser passada inteira como parâmetro de uma função ou procedimento:

```c
void mostrarPessoa(struct Pessoa p) {
    printf("%s tem %d anos\n", p.nome, p.idade);
}
```

## F6.1 A Ficha do Livro

Sua primeira struct

Uma biblioteca quer digitalizar o catálogo. Cada livro tem título, ano de publicação e preço, e essas três informações pertencem à mesma ficha.

> 💡 **Lembrete teórico:** Defina `struct Livro { char titulo[80]; int ano; float preco; };` antes do `main`. Para o campo de texto, use `strcpy` (lembre do `#include <string.h>`).

**Sua missão:**

1. Defina a struct `Livro` com os campos `titulo`, `ano` e `preco`.
2. No `main`, declare uma variável `livro1`, preencha os três campos (escolha os valores) e mostre tudo na tela.

EXEMPLO DE EXECUÇÃO

```
Titulo: O Pequeno Programador
Ano: 2021
Preco: R$ 39.90
```

## F6.2 A Ficha do Pet

Dois campos de texto na mesma struct

Uma clínica veterinária quer uma ficha simples para cada animal: nome, espécie e idade.

> 💡 **Lembrete teórico:** Uma struct pode ter mais de um campo de texto. Cada um precisa do seu próprio `strcpy`.

**Sua missão:**

1. Defina a struct `Animal` com os campos `nome`, `especie` (ambos texto) e `idade` (inteiro).
2. Preencha os dados de um pet (à sua escolha) e mostre a ficha completa.

EXEMPLO DE EXECUÇÃO

```
=== FICHA DO PET ===
Nome: Rex
Especie: Cachorro
Idade: 3 ano(s)
```

Try, Catch & Learn

## F6.3 O Caixa Registradora

struct lida do usuário + cálculo

Um pequeno comércio quer registrar a venda de um produto: nome, preço unitário e quantidade — e calcular o valor total da compra.

> 💡 **Lembrete teórico:** Para ler um texto com espaços (como "Caderno Grande") diretamente do usuário dentro de um campo de struct, use `scanf(" %[^\n]", item.nome);` em vez de `strcpy`.

**Sua missão:**

1. Defina a struct `Produto` com os campos `nome`, `preco` e `quantidade`.
2. Leia os três valores do usuário.
3. Calcule o total (`preco * quantidade`) e mostre a nota da compra.

EXEMPLO DE EXECUÇÃO

```
Nome do produto: Caderno
Preco unitario: 15.50
Quantidade: 3
--- Nota da compra ---
Caderno x3 = R$ 46.50
```

## F6.4 O Boletim da Turma

Vetor de struct

Uma professora quer um boletim para 3 alunos da turma, cada um com nome e duas notas, calculando a média de cada um no final.

> 💡 **Lembrete teórico:** Um vetor de struct se declara como `struct Aluno turma[3];`. Cada posição `turma[i]` é uma ficha completa, acessada normalmente com `turma[i].nome`, `turma[i].nota1`...

**Sua missão:**

1. Defina a struct `Aluno` com `nome`, `nota1` e `nota2`.
2. Declare o vetor `turma[3]` e, com um `for`, leia nome e as duas notas de cada aluno.
3. Com outro `for`, calcule e mostre a média de cada aluno.

EXEMPLO DE EXECUÇÃO

```
--- Aluno 1 ---
Nome: Ana
Nota 1: 8
Nota 2: 7
--- Aluno 2 ---
Nome: Bruno
Nota 1: 5
Nota 2: 6
--- Aluno 3 ---
Nome: Carla
Nota 1: 9
Nota 2: 10
=== BOLETIM ===
Ana - Media: 7.5
Bruno - Media: 5.5
Carla - Media: 9.5
```

Try, Catch & Learn

## F6.5 O Mapa do Tesouro

struct simples + função que recebe struct

Um mapa de tesouro marca pontos importantes com coordenadas x e y. Vamos criar uma struct Ponto e uma função que mostra qualquer ponto de forma padronizada.

> 💡 **Lembrete teórico:** Uma struct pode ser passada como parâmetro de uma função, assim como int ou float: `void mostrarPonto(struct Ponto p)`.

**Sua missão:**

1. Defina a struct `Ponto` com os campos x e y (inteiros).
2. Crie o procedimento `void mostrarPonto(struct Ponto p)`, que mostra `(x, y)`.
3. No `main`, crie dois pontos (origem e destino) com valores à sua escolha e mostre os dois usando o procedimento.

EXEMPLO DE EXECUÇÃO

```
Ponto de origem: (0, 0)
Ponto de destino: (5, 8)
```

## F6.6 A Garagem de Carros Antigos

Vetor de struct + procedimento

Um colecionador quer catalogar 3 carros de sua garagem: modelo, ano e preço, exibidos de forma padronizada.

> 💡 **Lembrete teórico:** Combine o que já viu: um vetor de struct (Questão 4) com um procedimento que recebe uma struct (Questão 5).

**Sua missão:**

1. Defina a struct `Carro` com `modelo`, `ano` e `preco`.
2. Declare o vetor `garagem[3]` e preencha os três carros diretamente no código (sem scanf, à sua escolha).
3. Crie o procedimento `void mostrarCarro(struct Carro c)` e use-o num `for` para exibir a garagem inteira.

EXEMPLO DE EXECUÇÃO

```
=== GARAGEM ===
Fusca (1978) - R$ 25000.00
Civic (2020) - R$ 98000.00
Gol (2015) - R$ 42000.00
```

## F6.7 O Duelo de Treinamento

**struct modificada por uma função (ponteiro)**

Um jogo de RPG precisa que a função de ataque altere de verdade a vida do personagem, e não apenas uma cópia dela. Para isso, vamos usar um conceito novo: passar o endereço da struct para a função, usando * e &.

> 💡 **Lembrete teórico:** Até aqui, toda struct passada para uma função era uma **cópia** — alterar os campos dentro da função não mudava o original. Para alterar de verdade, a função recebe um **ponteiro** para a struct (`struct Jogador *alvo`) e acessa os campos com `->` em vez de `.`. A chamada usa `&` para passar o endereço: `atacar(&heroi, 30);`.

**Sua missão:**

1. Defina a struct `Jogador` com `nome`, `vida` e `ataque`.
2. Crie o procedimento `void atacar(struct Jogador *alvo, int dano)`, que subtrai `dano` de `alvo->vida`.
3. No `main`, crie um jogador com 100 de vida, ataque-o duas vezes com valores diferentes de dano e mostre a vida atual após cada ataque.

EXEMPLO DE EXECUÇÃO

```
Cavaleiro comeca com 100 de vida.
Cavaleiro foi atacado! Vida atual: 70
Cavaleiro foi atacado de novo! Vida atual: 50
```

> 🔍 **Por que isso funciona?:** `alvo` guarda o endereço de memória de `heroi`, não uma cópia. Por isso, `alvo->vida = alvo->vida - dano;` altera a variável original no `main`. É a mesma ideia por trás do `&idade` que você já usa no `scanf` há várias fases — agora aplicada a uma struct inteira.

## F6.8 O Cálculo do Bônus

**Função com struct que devolve valor**

O setor de RH de uma empresa quer calcular automaticamente o bônus de um funcionário: 10% do salário.

> 💡 **Lembrete teórico:** Uma função pode receber uma struct e devolver um valor comum (`float`), sem precisar alterar a struct original — nesse caso, não precisamos de ponteiro, só de ler os campos.

**Sua missão:**

1. Defina a struct `Funcionario` com `nome` e `salario`.
2. Crie a função `float calcularBonus(struct Funcionario f)`, que devolve `f.salario * 0.10`.
3. No `main`, leia os dados de um funcionário, calcule o bônus com a função e mostre o resultado.

EXEMPLO DE EXECUÇÃO

```
Nome do funcionario: Mariana
Salario: 2500
Mariana recebera um bonus de R$ 250.00
```

## F6.9 A Nota Fiscal do Mercado

Vetor de struct + total acumulado

Um mini mercado quer emitir uma nota fiscal simples para 4 produtos do carrinho, somando o total da compra.

> 💡 **Lembrete teórico:** Combine tudo que já viu: vetor de struct, percorrido com `for`, acumulando um total — o mesmo padrão da Questão 3 da Fase 5, agora com cada item sendo uma ficha completa em vez de um número solto.

**Sua missão:**

1. Defina a struct `Produto` com `nome` e `preco`.
2. Declare o vetor `carrinho[4]` e preencha os 4 produtos diretamente no código.
3. Com um `for`, mostre cada item (nome e preço) e some o total.
4. Mostre o total da nota fiscal.

EXEMPLO DE EXECUÇÃO

```
=== NOTA FISCAL ===
Arroz      R$ 22.50
Feijao     R$ 8.90
Leite      R$ 4.50
Pao        R$ 7.20
-------------------------
TOTAL: R$ 43.10
```

## F6.10 A Apuração da Eleição

Vetor de struct + busca do maior

Uma eleição estudantil tem 3 candidatos. O sistema precisa ler os votos de cada um e anunciar o vencedor.

> 💡 **Lembrete teórico:** Combine a busca do maior valor (Questão 4 da Fase 5) com struct: em vez de guardar só o maior valor, guarde o índice de quem está vencendo (`maiorIndice`), para poder mostrar o nome do candidato no final.

**Sua missão:**

1. Defina a struct `Candidato` com `nome` e `votos`.
2. Declare o vetor `eleicao[3]`, já com os nomes preenchidos no código, e leia os votos de cada um.
3. Encontre o índice do candidato com mais votos (mesmo padrão de "maior valor", mas guardando a posição).
4. Mostre o nome e os votos do(a) vencedor(a).

EXEMPLO DE EXECUÇÃO

```
Votos para Ana: 120
Votos para Bruno: 340
Votos para Carla: 210
=== RESULTADO ===
Vencedor(a): Bruno com 340 votos!
```

# DESAFIOS FINAIS

Agora é hora de juntar vetores, matrizes, structs, laços e funções. Estes desafios são mais longos que as questões, então planeje no papel antes de digitar. Faça por partes, teste cada parte e só depois avance.

## D1 A Caravana de Mercadores

Desafio 1 · Vetor + estatísticas completas

Uma caravana carrega 10 itens de pesos diferentes. O chefe da caravana precisa de um relatório completo: peso total, peso médio, o item mais pesado, o mais leve, e um alerta se o peso total ultrapassar o limite seguro de 500 kg.

**Sua missão:**

1. Declare o vetor `caravana[10]` e leia os 10 pesos.
2. Num único `for`, calcule a soma, e também atualize o maior e o menor peso encontrados (sem usar laços separados para cada estatística).
3. Calcule a média (atenção: para um resultado com casas decimais, converta a soma para `float` antes de dividir, usando `(float) soma / 10`).
4. Mostre o relatório completo e o alerta, se aplicável.

EXEMPLO DE EXECUÇÃO

```
=== Exemplo: caravana acima do limite ===
Peso do item 1 (kg): 80
Peso do item 2 (kg): 45
Peso do item 3 (kg): 60
Peso do item 4 (kg): 30
Peso do item 5 (kg): 55
Peso do item 6 (kg): 40
Peso do item 7 (kg): 70
Peso do item 8 (kg): 25
Peso do item 9 (kg): 50
Peso do item 10 (kg): 65
--- Relatorio da caravana ---
Peso total: 520 kg
Peso medio: 52.00 kg
Item mais pesado: 80 kg
Item mais leve: 25 kg
ALERTA: caravana acima do limite de 500 kg!

=== Exemplo: caravana dentro do limite ===
Peso do item 1 (kg): 10
Peso do item 2 (kg): 20
Peso do item 3 (kg): 15
Peso do item 4 (kg): 25
Peso do item 5 (kg): 30
Peso do item 6 (kg): 5
Peso do item 7 (kg): 18
Peso do item 8 (kg): 22
Peso do item 9 (kg): 12
Peso do item 10 (kg): 8
--- Relatorio da caravana ---
Peso total: 165 kg
Peso medio: 16.50 kg
Item mais pesado: 30 kg
Item mais leve: 5 kg
Peso dentro do limite seguro.
```

💡 **Pista:** `(float) soma / 10` é um cast: converte `soma` para decimal só para esta conta, sem mudar o tipo da variável `soma` em si. Sem o cast, `soma / 10` faria divisão inteira e perderia as casas decimais.

## D2 O Jogo da Velha Matemático

Desafio 2 · Matriz 3×3 + funções + diagonais

Um professor de matemática quer um "jogo da velha" especial: em vez de X e O, o tabuleiro 3×3 é preenchido com números, e o programa calcula a soma de cada diagonal.

> 💡 **Lembrete teórico:** A diagonal principal de uma matriz quadrada é onde `linha == coluna` (`m[i][i]`). A diagonal secundária, para uma matriz 3×3, é onde a soma dos índices dá 2 (`m[i][2-i]`) — o mesmo princípio do Desafio 1 da Fase 3, agora aplicado a somas em vez de desenho.

**Sua missão:**

1. Crie o procedimento `void preencherMatriz(int m[3][3])`, que lê os 9 valores com dois `for` aninhados.
2. Crie a função `int somarDiagonalPrincipal(int m[3][3])`, que soma `m[i][i]` para `i` de 0 a 2.
3. Crie a função `int somarDiagonalSecundaria(int m[3][3])`, que soma `m[i][2-i]`.
4. Crie o procedimento `void exibirMatriz(int m[3][3])`, que mostra o tabuleiro formatado.
5. No `main`, declare a matriz, preencha, exiba e mostre as duas somas.

EXEMPLO DE EXECUÇÃO

```
=== PREENCHENDO O TABULEIRO 3x3 ===
Valor da casa [0][0]: 1
Valor da casa [0][1]: 2
Valor da casa [0][2]: 3
Valor da casa [1][0]: 4
Valor da casa [1][1]: 5
Valor da casa [1][2]: 6
Valor da casa [2][0]: 7
Valor da casa [2][1]: 8
Valor da casa [2][2]: 9
--- Tabuleiro ---
 1  2  3
 4  5  6
 7  8  9
Soma da diagonal principal: 15
Soma da diagonal secundaria: 15
```

## D3 O Cofre de Quatro Pinos

Desafio 3 · Dois vetores comparados + laço com break

Um cofre de segurança tem uma senha secreta de 4 pinos (dígitos de 0 a 9). O visitante tem até 5 tentativas; a cada uma, o cofre informa quantos pinos acertou na posição certa (como num jogo de adivinhação).

| Fonte | O que guarda |
|---|---|
| `senha[4]` | A senha secreta, fixa no código |
| `tentativa[4]` | O palpite do visitante em cada rodada |

**Sua missão:**

1. Declare `senha[4]` com os valores `{3, 7, 1, 9}` (fixos no código).
2. Use um `for` de até 5 rodadas. Em cada uma: leia 4 valores no vetor `tentativa`, compare posição a posição com `senha`, contando os acertos.
3. Mostre quantos pinos acertou. Se acertar os 4, mostre `COFRE ABERTO!` e pare o laço com `break`.
4. Se as 5 tentativas acabarem sem acerto total, mostre que o cofre permanece fechado.

EXEMPLO DE EXECUÇÃO

```
=== Exemplo: acerto na 3ª tentativa ===
=== COFRE DE 4 PINOS ===
Tentativa 1 de 5
Pino 1 (0-9): 1
Pino 2 (0-9): 2
Pino 3 (0-9): 3
Pino 4 (0-9): 4
Pinos corretos na posicao certa: 0 de 4
Tentativa 2 de 5
Pino 1 (0-9): 3
Pino 2 (0-9): 7
Pino 3 (0-9): 1
Pino 4 (0-9): 5
Pinos corretos na posicao certa: 3 de 4
Tentativa 3 de 5
Pino 1 (0-9): 3
Pino 2 (0-9): 7
Pino 3 (0-9): 1
Pino 4 (0-9): 9
Pinos corretos na posicao certa: 4 de 4
COFRE ABERTO!

=== Exemplo: 5 tentativas sem acerto ===
=== COFRE DE 4 PINOS ===
Tentativa 1 de 5
Pino 1 (0-9): 0
Pino 2 (0-9): 0
Pino 3 (0-9): 0
Pino 4 (0-9): 0
Pinos corretos na posicao certa: 0 de 4
...
Tentativas esgotadas! O cofre permanece fechado.
```

💡 **Pista:** A comparação precisa ser **posição a posição**: `tentativa[0]` só conta como acerto se for igual a `senha[0]`, e assim por diante. Não vale comparar se o dígito existe em qualquer posição da senha, só

se está no lugar certo.

# 🏆 Placar de Conquistas

Marque as missões que você já completou. Não vale pular a parte de testar seu programa com valores diferentes!

| Missão | Tema | Concluída |
|---|---|---|
| **F5.1** | As Caixas Numeradas — vetor + for | [ ] |
| **F5.2** | A Mão de Cartas — vetor pré-preenchido | [ ] |
| **F5.3** | O Tesouro dos Oito Baús — vetor + acumulador | [ ] |
| **F5.4** | O Recorde do Campeonato — maior valor | [ ] |
| **F5.5** | A Semana Mais Fria — menor valor | [ ] |
| **F5.6** | A Despensa Mágica — busca + break | [ ] |
| **F5.7** | Calculadora de Médias — vetor em função | [ ] |
| **F5.8** | Relatório do Depósito — vetor em procedimento | [ ] |
| **F5.9** | O Tabuleiro Misterioso — matriz 2×2 | [ ] |
| **F5.10** | O Mapa do Cinema — matriz + contagem | [ ] |
| **F6.1** | A Ficha do Livro — primeira struct | [ ] |
| **F6.2** | A Ficha do Pet — struct com 2 textos | [ ] |
| **F6.3** | O Caixa Registradora — struct + scanf | [ ] |
| **F6.4** | O Boletim da Turma — vetor de struct | [ ] |
| **F6.5** | O Mapa do Tesouro — struct em função | [ ] |
| **F6.6** | Garagem de Carros — vetor de struct + proc. | [ ] |
| **F6.7** | O Duelo de Treinamento — struct + ponteiro | [ ] |
| **F6.8** | Cálculo do Bônus — função com retorno | [ ] |
| **F6.9** | Nota Fiscal do Mercado — vetor de struct + total | [ ] |
| **F6.10** | Apuração da Eleição — vetor de struct + maior | [ ] |
| **D1** | A Caravana de Mercadores — vetor + estatísticas | [ ] |
| **D2** | Jogo da Velha Matemático — matriz + diagonais | [ ] |
| **D3** | Cofre de Quatro Pinos — vetores comparados | [ ] |

## 🎉 Missão cumprida!

Se você chegou até aqui, já sabe organizar grandes quantidades de dados em vetores e matrizes, e agrupar informações relacionadas em `struct`. Com essas ferramentas, seu código consegue representar praticamente qualquer coisa do mundo real: uma turma de alunos, um estoque, um time de jogadores. Continue praticando: tente combinar vetores de struct com tudo que já aprendeu nas fases anteriores!